

Object Oriented Programming Introduction

Lecture No. 1

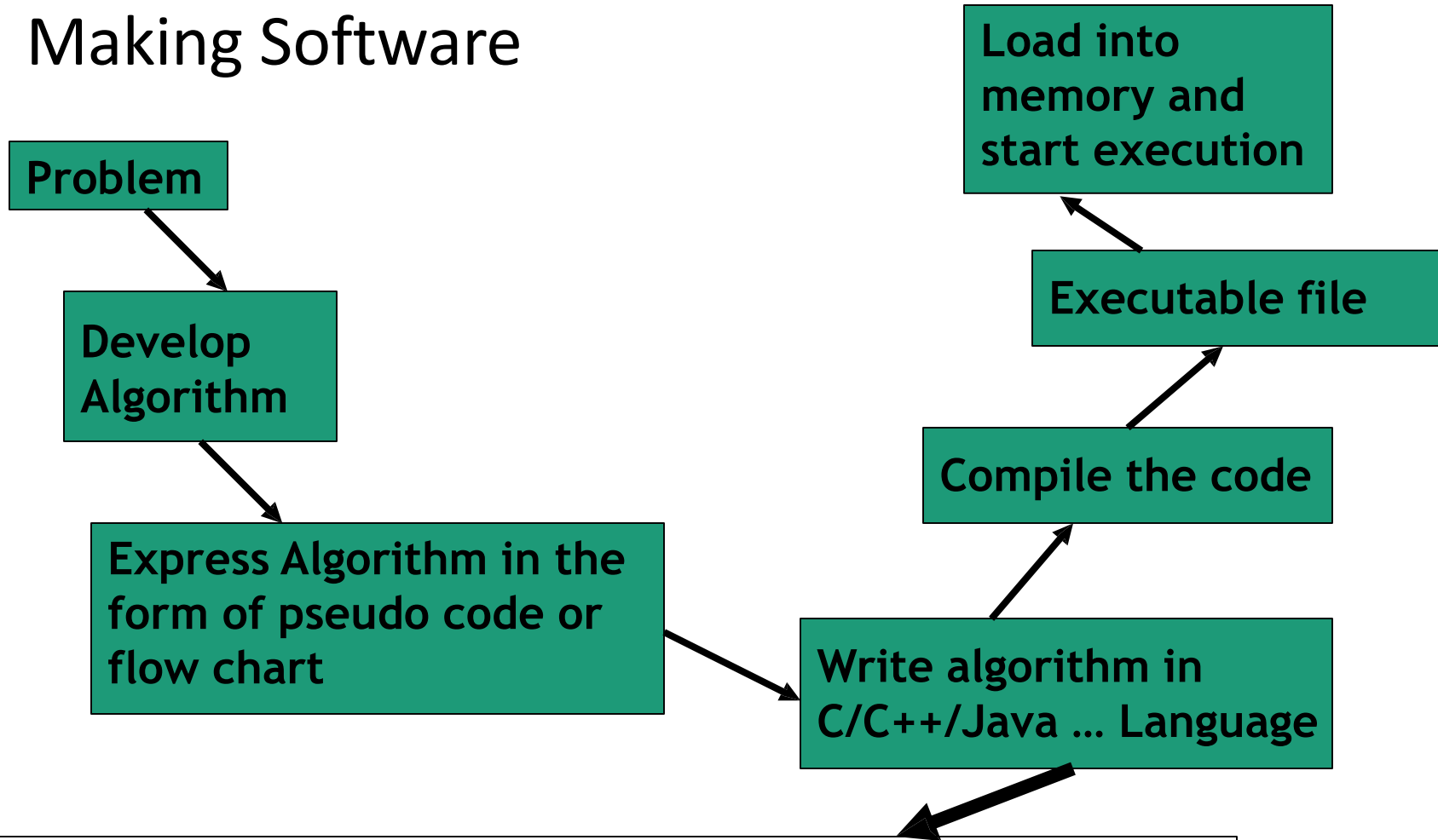
Lecture Contents

- Introduction
- C++ Programming basics
- Pointers
- Functions
- Structures
- Classes (OOP basics, inheritance, ...)
- Operator overloading
- Streams and Files
- Templates and Exceptions

What is a Computer?

- Computer
 - A device capable of performing computations and making logical decisions
- Computer programs
 - Sets of instructions that control a computer's processing of data
- Hardware
 - Various devices comprising a computer
 - Examples: keyboard, screen, mouse, disks, memory, CD-ROM, and processing units
- Software
 - Programs that run a computer

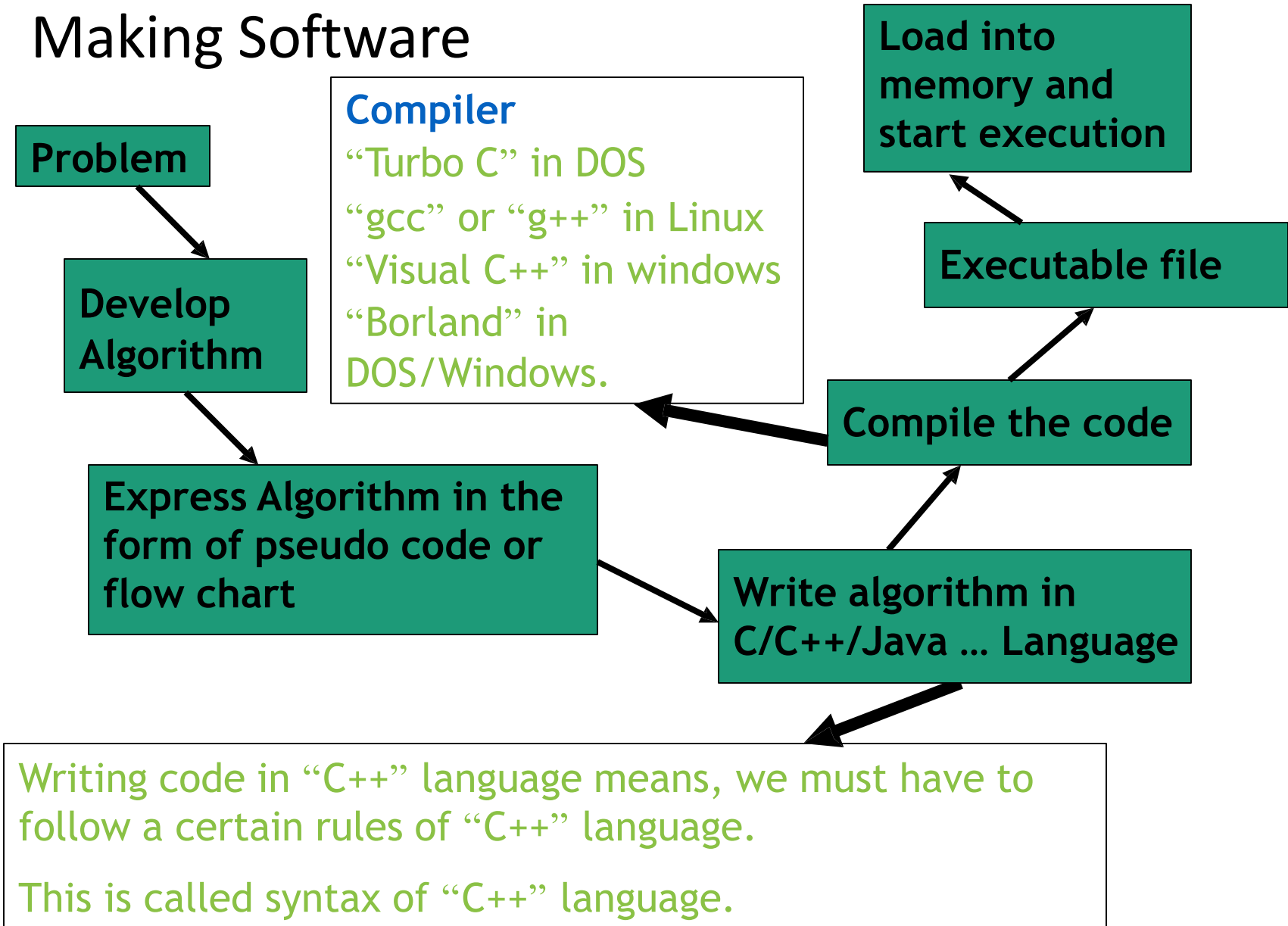
Making Software



Writing code in “C++” language means, we must have to follow a certain rules of “C++” language.

This is called syntax of “C++” language.

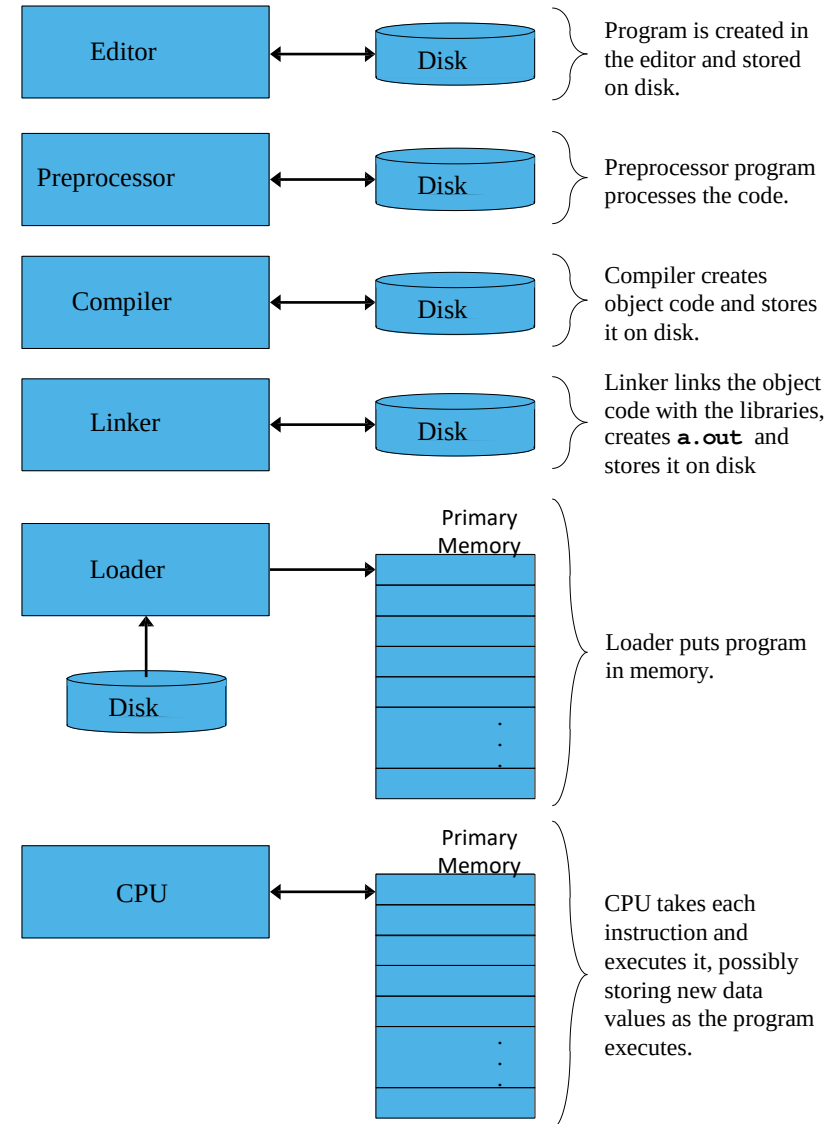
Making Software



Basics of a Typical C++ Environment

Phases of C++ Programs:

1. Edit
2. Preprocess
3. Compile
4. Link
5. Load
6. Execute



Introduction to C++ Programming

- C++ language
 - Facilitates a structured and disciplined approach to computer program design
- There are various C++ standards:
 - C++98: First ISO standard
 - C++11 (aka C++0x) : Major Enhancement to language and library
 - C++14 (aka C++1y): Bug fixes and improvements to C++
 - C++17 (aka C++1z): Library additions and Bug fixes
 - C++2a (next planned standard in 202x)
- Following are several examples
 - The examples illustrate many important features of C++
 - Each example is analyzed one statement at a time.

```
1 // Fig. 1.2: fig01_02.cpp
2 // A first program in C++
3 #include <iostream>
4
5 int main()
6 {
7     std::cout << "Welcome to C++!\n";
8
9     return 0; // indicate that program terminated normally
10 }
```

Comments

Written between `/*` and `*/` or following a `//`.

Improve program readability and do not cause the computer to perform any action.

preprocessor directive

Message to the C++ preprocessor.

Lines beginning with `#` are preprocessor directives.

`#include <iostream>` tells the preprocessor to

C++ programs contain one or more functions, one of which must be **main**

Parenthesis are used to indicate a function

Welcome to C++!

Prints the *string* of characters contained between the

return is a way to exit a function from a function.

return 0, in this case, means that the program terminated normally.

including `std::cout`, the `<<` `g "Welcome to C++!\n"` and `)`, is called a *statement*.

All statements must end with a semicolon.

integer value.

function

A Simple Program: Printing a Line of Text

- **std::cout**
 - Standard output stream object
 - “Connected” to the screen
 - **std::** specifies the "namespace" which **cout** belongs to
 - **std::** can be removed through the use of **using** statements
- **<<**
 - Stream insertion operator
 - Value to the right of the operator (right operand) inserted into output stream (which is connected to the screen)
 - **std::cout << “Welcome to C++!\n”;**
- ****
 - Escape character
 - Indicates that a “special” character is to be output

A Simple Program: Printing a Line of Text

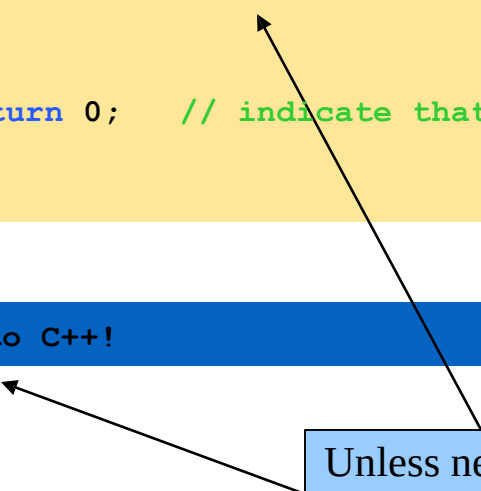
Escape Sequence	Description
<code>\n</code>	Newline. Position the screen cursor to the beginning of the next line.
<code>\t</code>	Horizontal tab. Move the screen cursor to the next tab stop.
<code>\r</code>	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
<code>\a</code>	Alert. Sound the system bell.
<code>\\</code>	Backslash. Used to print a backslash character.
<code>\"</code>	Double quote. Used to print a double quote character.

- There are multiple ways to print text
 - Following are more examples

```
1 // Fig. 1.4: fig01_04.cpp
2 // Printing a line with multiple statements
3 #include <iostream>
4
5 int main()
6 {
7     std::cout << "Welcome ";
8     std::cout << "to C++!\n";
9
10    return 0;    // indicate that program ended successfully
11 }
```

Welcome to C++!

Unless new line '**\n**' is specified, the text continues on the same line.



Another Simple Program: Adding Two Integers

- Variables

- Location in memory where a value can be stored for use by a program
- Must be declared with a name and a data type before they can be used
- Some common data types are:
 - `int` - integer numbers
 - `char` - characters
 - `double` - floating point numbers
- Example: `int myvariable;`
 - Declares a variable named `myvariable` of type `int`
- Example: `int variable1, variable2;`
 - Declares two variables, each of type `int`

Another Simple Program: Adding Two Integers

- `>>` (stream extraction operator)
 - When used with `std::cin`, waits for the user to input a value and stores the value in the variable to the right of the operator
 - The user types a value, then presses the *Enter* (Return) key to send the data to the computer

- Example:

```
int myVariable;  
std::cin >> myVariable;
```

- Waits for user input, then stores input in `myVariable`

- `=` (assignment operator)
 - Assigns value to a variable
 - Binary operator (has two operands)
- Example:

```
sum = variable1 + variable2;
```

```

1 // Fig. 1.6: fig01_06.cpp
2 // Addition program
3 #include <iostream>
4
5 int main()
6 {
7     int integer1, integer2, sum;           // declaration
8
9     std::cout << "Enter first integer\n"; //
10    std::cin >> integer1; ← //
11    std::cout << "Enter second integer\n"; //
12    std::cin >> integer2;                 // read an integer
13    sum = integer1 + integer2;             // assignment of sum
14    std::cout << "Sum is " << sum << std::endl; // print sum
15
16    return 0; // indicate that program ended succe
17 }

```

Notice how **std::cin** is used to get user input.

std::endl flushes the buffer and prints a newline.

Variables can be output using **std::cout << variableName**.

```

Enter first integer
45
Enter second integer
72
Sum is 117

```

Arithmetic

C++ o p e r a t i o n	Arithm etic o p e r a t o r	Alg e b r a i c exp r e s s i o n	C++ exp r e s s i o n
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm	<code>b * m</code>
Division	/	x / y	<code>x / y</code>
Modulus	%	$r \bmod s$	<code>r % s</code>

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they are evaluated left to right.
*, /, or %	Multiplication Division Modulus	Evaluated second. If there are several, they are evaluated left to right.
+ or -	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Decision Making: Equality/Relational Operators

- **using** statements

- Eliminate the need to use the **std::** prefix
- Allow us to write **cout** instead of **std::cout**
- To use the following functions without the **std::** prefix, write the following at the top of the program

```
using std::cout;
```

```
using std::cin;
```

```
using std::endl;
```



```

1 // Fig. 1.14: fig01_14.cpp
2 // Using if statements, relational
3 // operators, and equality operators
4 #include <iostream>
5
6 using std::cout; // program uses cout
7 using std::cin;  // program uses cin
8 using std::endl; // program uses endl
9
10 int main()
11 {
12     int num1, num2;
13
14     cout << "Enter two integers, and I will tell you\n"
15           << "the relationships they satisfy: ";
16     cin >> num1 >> num2; // read
17
18     if ( num1 == num2 )
19         cout << num1 << " is equal to " << num2 << endl;
20
21     if ( num1 != num2 )
22         cout << num1 << " is not equal to " << num2 << endl;
23
24     if ( num1 < num2 )
25         cout << num1 << " is less than " << num2 << endl;
26
27     if ( num1 > num2 )
28         cout << num1 << " is greater than " << num2 << endl;
29
30     if ( num1 <= num2 )
31         cout << num1 << " is less than or equal to "
32               << num2 << endl;
33

```

Notice the **using** statements.

Enter two integers, and I will tell you
the relationships they satisfy: 3 7

The **if** statements test the truth
of the condition. If it is **true**,

3 is not equal to 7
is
executed. If not, body is
skipped.

3 is less than 7

statements
in a body, delineate them with
braces { }.

3 is less than or equal to 7

```
34     if ( num1 >= num2 )
35         cout << num1 << " is greater than or equal to "
36             << num2 << endl;
37
38     return 0;    // indicate that program ended successfully
39 }
```

```
Enter two integers, and I will tell you
the relationships they satisfy: 3 7
3 is not equal to 7
3 is less than 7
3 is less than or equal to 7
```

```
Enter two integers, and I will tell you
the relationships they satisfy: 22 12
22 is not equal to 12
22 is greater than 12
22 is greater than or equal to 12
```

```
Enter two integers, and I will tell you
the relationships they satisfy: 7 7
7 is equal to 7
7 is less than or equal to 7
7 is greater than or equal to 7
```

References

- Course slides – based on course slides prepared by Dr. Shariq Bashir and Dr. Uzair Khan, and Dr. Ehatsham CP Spring 2011-2012.
- **Text Books:**
 - “Object Oriented Programming in C++”, Author (s): Robert Lafore.
 - “C++ How to program”, Author (s): M. Deitel & Dietel.
 - “Turbo C Programming”, Author (s): Robert Lafore.

- Questions?